AUNG KO KO OO

Founder & Lead Researcher

Burmese NLP Research Institute

MACHINE LEARNING
Complete Course

Logic, Theory, and Equations

From Beginner to Advanced

Published by:
Burmese NLP Research Institute
Yangon, Myanmar
AungKoKoOo@my.uopeople.edu
www.burmese-nlp-research.org

May 7, 2026

© 2025 Burmese NLP Research Institute. All Rights Reserved.
No part of this publication may be reproduced without permission.

About the Author

AUNG KO KO OO is the Founder and Lead Researcher at the Burmese NLP Research Institute. He holds advanced degrees in Artificial Intelligence and Machine Learning from leading universities.

His research focuses on:

- Natural Language Processing for Burmese Language
- Deep Learning Architectures
- Low-Resource Machine Translation
- Speech Recognition for Myanmar Languages

He has published numerous papers in international conferences and journals, and has been teaching Machine Learning for over 8 years.

Important

Contact Information:

- Email: AungKoKoOo@my.uopeople.edu or www.burmese-nlp-research.org
- GitHub: https://github.com/sage2029
- LinkedIn: https://www.linkedin.com/in/aung-ko-ko-oo-ab3a9839a/

Contents

About the Author **3**

1 Fundamental Concepts of Machine Learning **9**
1.1 What is Machine Learning? . . . 9
1.1.1 Traditional Programming vs Machine Learning . . . 9
1.1.2 Three Types of Machine Learning . . . 9
1.2 ML Life Cycle (5 Stages) . . . 10
1.3 Basic Terminology . . . 10

2 Linear Regression **11**
2.1 Simple Linear Regression . . . 11
2.2 Multiple Linear Regression . . . 11
2.3 Mean Squared Error (MSE) - Loss Function . . . 11
2.3.1 Mean Absolute Error (MAE) . . . 12
2.3.2 Root Mean Squared Error (RMSE) . . . 12

3 Gradient Descent **13**
3.1 Fundamental Concept of Gradient Descent . . . 13
3.2 Derivative of the Loss Function . . . 13
3.3 Choosing the Learning Rate (α) . . . 13
3.4 Types of Gradient Descent . . . 14
3.5 Normal Equation (Direct Solution) . . . 14

4 Logistic Regression **15**
4.1 Why Not Linear Regression for Classification? . . . 15
4.2 Sigmoid Function (Logistic Function) . . . 15
4.3 Logistic Regression Hypothesis . . . 15
4.4 Decision Boundary . . . 15
4.5 Log Loss (Cross-Entropy Loss) . . . 16

5 Regularization (Preventing Overfitting) **17**
5.1 Fundamental Concept of Regularization . . . 17
5.2 Ridge Regression (L2 Regularization) . . . 17
5.3 Lasso Regression (L1 Regularization) . . . 18
5.4 Elastic Net (L1 + L2) . . . 18
5.5 When to Use Which Regularization? . . . 18

6 Neural Networks **19**
6.1 Mathematical Model of a Neuron (Perceptron) . . . 19
6.2 Activation Functions . . . 19
6.2.1 Sigmoid . . . 19
6.2.2 Tanh (Hyperbolic Tangent) . . . 19
6.2.3 ReLU (Rectified Linear Unit) - Most Popular . . . 20
6.2.4 Leaky ReLU . . . 20
6.2.5 Softmax (Multi-class Classification) . . . 20
6.3 Forward Propagation . . . 20
6.4 Backpropagation (Learning from Errors) . . . 20
6.5 Loss Functions for Neural Networks . . . 21

7 Bias-Variance Tradeoff **23**
7.1 Understanding Bias and Variance . . . 23
7.2 Underfitting vs Overfitting . . . 23
7.3 How to Diagnose . . . 24
7.4 Finding the Sweet Spot . . . 24

8 Unsupervised Learning **25**
8.1 K-Means Clustering . . . 25
8.1.1 Objective Function . . . 25
8.1.2 Euclidean Distance . . . 25
8.1.3 K-Means Algorithm . . . 25
8.1.4 Choosing K - Elbow Method . . . 25
8.2 Principal Component Analysis (PCA) . . . 26
8.2.1 Purpose of PCA . . . 26
8.2.2 PCA Algorithm . . . 26
8.2.3 Variance Explained . . . 26

9 Model Evaluation Metrics **27**
9.1 Confusion Matrix . . . 27
9.2 Classification Metrics . . . 27
9.2.1 Accuracy . . . 27
9.2.2 Precision . . . 27
9.2.3 Recall (Sensitivity) . . . 27
9.2.4 F1-Score (Harmonic Mean) . . . 27
9.2.5 Specificity . . . 28
9.3 Regression Metrics . . . 28
9.3.1 Mean Squared Error (MSE) . . . 28
9.3.2 Root Mean Squared Error (RMSE) . . . 28
9.3.3 Mean Absolute Error (MAE) . . . 28
9.3.4 R-squared (Coefficient of Determination) . . . 28
9.3.5 Adjusted R-squared (for multiple features) . . . 29

10 Complete Summary - Quick Reference Card **31**
10.1 Linear Regression . . . 31
10.2 Logistic Regression . . . 31
10.3 Gradient Descent . . . 31
10.4 Regularization . . . 31

10.5 Neural Networks 32
10.6 Evaluation Metrics - Quick Reference 32
10.7 Unsupervised Learning 32

11 Appendix: Common Machine Learning Algorithms 33
11.1 Algorithm Cheat Sheet 33

Publication Information 35

References 37

1 Fundamental Concepts of Machine Learning

1.1 What is Machine Learning?

Definition

Machine Learning (ML) is a field of artificial intelligence that enables computers to learn patterns from data without being explicitly programmed with rules.

1.1.1 Traditional Programming vs Machine Learning

Equation

$$\begin{aligned} \text{Traditional Programming:} \quad & \text{Data} + \text{Rules} \longrightarrow \text{Output} \\ \text{Machine Learning:} \quad & \text{Data} + \text{Output} \longrightarrow \text{Rules (Model)} \end{aligned}$$

1.1.2 Three Types of Machine Learning

1. **Supervised Learning:**
   - Data includes labels (output)
   - Regression and Classification
   - Example: House price prediction, Spam detection
2. **Unsupervised Learning:**
   - Data has no labels
   - Clustering and Dimensionality Reduction
   - Example: Customer segmentation, Anomaly detection
3. **Reinforcement Learning:**
   - Learning through rewards and penalties
   - Agent interacts with environment
   - Example: Game playing, Robotics, Self-driving cars

1.2 ML Life Cycle (5 Stages)

Data Collection
↓
Data Preprocessing
↓
Model Training
↓
Model Evaluation
↓
Deployment

Figure 1.1: Stages of an ML Project

1.3 Basic Terminology

Definition

- **Feature** (x) - Input variable used for prediction
- **Label** (y) - Target variable we want to predict
- **Training Set** - Data used to train the model (70-80%)
- **Test Set** - Data used to evaluate the model (20-30%)
- **Model** (h_θ) - Mathematical function that makes predictions

Example

Real-world Example: Housing Price Prediction

- Features (x): Size (sq ft), Number of bedrooms, Location score
- Label (y): House price in lakhs
- Training: Model learns from 1000 houses with known prices
- Testing: Model predicts prices for 200 new houses
- Goal: Accurately predict prices of unseen houses

2 Linear Regression

2.1 Simple Linear Regression

Theorem

Simple Linear Regression Form:

$$\boxed{h_\theta(x) = \theta_0 + \theta_1 x_1} \tag{2.1}$$

- θ_0 (Intercept) - The value of $h_\theta(x)$ when $x = 0$
- θ_1 (Slope) - The rate of change of $h_\theta(x)$ with respect to x

Example

House Price Prediction Example:
Price = 20 + 0.05 × (Size in sq ft)
For a 1000 sq ft house: Price = 20 + 50 = 70 (lakhs)

2.2 Multiple Linear Regression

Equation

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \tag{2.2}$$

$$= \sum_{j=0}^{n} \theta_j x_j \quad \text{where } x_0 = 1 \tag{2.3}$$

In vector form:

$$\boxed{h_\theta(x) = \theta^T x} \tag{2.4}$$

2.3 Mean Squared Error (MSE) - Loss Function

Definition

MSE measures the average squared difference between predicted and actual values.

Equation

$$\boxed{J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2}$$

Important

Notation Meaning:

$$\begin{aligned} m &= \text{Number of training examples} \\ h_\theta(x^{(i)}) &= \text{Prediction for } i\text{-th example} \\ y^{(i)} &= \text{Actual value for } i\text{-th example} \end{aligned}$$

2.3.1 Mean Absolute Error (MAE)

$$\boxed{\text{MAE} = \frac{1}{m}\sum_{i=1}^{m}|h_\theta(x^{(i)}) - y^{(i)}|} \tag{2.5}$$

2.3.2 Root Mean Squared Error (RMSE)

$$\boxed{\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2}} \tag{2.6}$$

3 Gradient Descent

3.1 Fundamental Concept of Gradient Descent

Theorem

Gradient Descent is an optimization algorithm that finds the parameters θ that minimize the cost function $J(\theta)$.

Equation

$$\boxed{\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)}$$

Important

Symbol Meaning:

$:=$	Assignment operator (update value)
α	Learning Rate (step size, $0 < \alpha \leq 1$)
$\frac{\partial J}{\partial \theta}$	Partial Derivative (slope of the cost function)

3.2 Derivative of the Loss Function

For Linear Regression:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)} \tag{3.1}$$

Therefore, the Update Rule becomes:

$$\boxed{\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}} \tag{3.2}$$

3.3 Choosing the Learning Rate (α)

1. **Too Small Learning Rate:**

- Very slow convergence
- Requires many iterations

2. **Too Large Learning Rate:**
   - May overshoot the minimum
   - May cause divergence (fail to converge)
3. **Optimal Learning Rate:**
   - Test values like: 0.01, 0.03, 0.1, 0.3, 1.0
   - Monitor cost function over iterations

3.4 Types of Gradient Descent

1. **Batch Gradient Descent:**
   - Uses all m training examples per update
   - Accurate but slow for large datasets
   - $O(m)$ per iteration
2. **Stochastic Gradient Descent (SGD):**
   - Uses one training example per update ($m = 1$)
   - Fast but noisy updates
   - May oscillate around minimum
3. **Mini-Batch Gradient Descent:**
   - Uses b examples per update (e.g., 32, 64, 128)
   - Best of both worlds: fast and stable
   - Most commonly used in practice

3.5 Normal Equation (Direct Solution)

Equation

$$\boxed{\theta = (X^T X)^{-1} X^T y}$$

Important

Advantages and Disadvantages:

+ No need to choose α or iterate

- Computing $(X^T X)^{-1}$ is $O(n^3)$ time complexity

- Not recommended when number of features $n > 1000$

4 Logistic Regression

4.1 Why Not Linear Regression for Classification?

Linear regression outputs values in $(-\infty, +\infty)$, but classification requires outputs in $\{0, 1\}$. The sigmoid function solves this problem.

4.2 Sigmoid Function (Logistic Function)

Equation

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

Important

Properties of Sigmoid:

$$\begin{aligned} \lim_{z \to +\infty} \sigma(z) &= 1 \\ \lim_{z \to -\infty} \sigma(z) &= 0 \\ \sigma(0) &= 0.5 \quad \text{(Decision boundary)} \end{aligned}$$

4.3 Logistic Regression Hypothesis

Equation

$$\boxed{h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}}$$

Interpretation:

$$h_\theta(x) = P(y = 1 \mid x; \theta) \tag{4.1}$$

4.4 Decision Boundary

- $\theta^T x = 0$ defines the decision boundary

- **Linear Boundary:** $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$
- **Non-linear Boundary:** $\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 = 0$

4.5 Log Loss (Cross-Entropy Loss)

Theorem

For Binary Classification:

$$\boxed{J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right]} \tag{4.2}$$

Important

Intuition behind Log Loss:

$$\begin{aligned} \text{If } y = 1: &\quad \text{Loss} = -\log(h) \\ \text{If } y = 0: &\quad \text{Loss} = -\log(1-h) \end{aligned}$$

- When prediction is correct: Loss is small
- When prediction is wrong: Loss is large
- As $h \to 0$ when $y = 1$: Loss $\to +\infty$ (heavy penalty)

5 Regularization (Preventing Overfitting)

5.1 Fundamental Concept of Regularization

Definition

Regularization adds a penalty term to the loss function to discourage large coefficients, preventing overfitting.

$$\boxed{J(\theta) = \text{Original Loss} + \lambda \times \text{Penalty}(\theta)} \tag{5.1}$$

Important

λ **(Lambda) = Regularization Strength**

$$\begin{aligned} \lambda = 0 \quad &\rightarrow \quad \text{No regularization (may overfit)} \\ \lambda = \infty \quad &\rightarrow \quad \theta \approx 0 \text{ (underfitting)} \end{aligned}$$

5.2 Ridge Regression (L2 Regularization)

Equation

$$\boxed{J(\theta) = \text{MSE} + \lambda \sum_{j=1}^{n} \theta_j^2}$$

- Shrinks all coefficients toward zero
- Does not make coefficients exactly zero
- Good when all features are potentially useful

5.3 Lasso Regression (L1 Regularization)

Equation

$$J(\theta) = \text{MSE} + \lambda \sum_{j=1}^{n} |\theta_j|$$

- Can make some coefficients exactly zero
- Performs automatic feature selection
- Good when many features are irrelevant

5.4 Elastic Net (L1 + L2)

Equation

$$J(\theta) = \text{MSE} + \lambda_1 \sum_{j=1}^{n} |\theta_j| + \lambda_2 \sum_{j=1}^{n} \theta_j^2$$

5.5 When to Use Which Regularization?

Regularization	**When to Use**
Ridge (L2)	When all features are potentially useful, but want to keep coefficients small
Lasso (L1)	When many features are irrelevant and you want feature selection
Elastic Net	When number of features exceeds observations, or features are correlated

Table 5.1: Choosing Regularization Method

6 Neural Networks

6.1 Mathematical Model of a Neuron (Perceptron)

Equation

$$z = \sum_{i=1}^{n} w_i x_i + b = W^T X + b \tag{6.1}$$

$$a = f(z) \tag{6.2}$$

Important

Symbol Definitions:

$$\begin{aligned} x_i &= \text{Input (feature value)} \\ w_i &= \text{Weight (importance of connection)} \\ b &= \text{Bias (threshold for activation)} \\ z &= \text{Weighted sum} \\ f &= \text{Activation function} \\ a &= \text{Activation (output of neuron)} \end{aligned}$$

6.2 Activation Functions

6.2.1 Sigmoid

$$\boxed{f(z) = \frac{1}{1 + e^{-z}}}, \quad \text{Output range: } (0, 1) \tag{6.3}$$

Use for: Binary classification output layer

6.2.2 Tanh (Hyperbolic Tangent)

$$\boxed{f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}}, \quad \text{Output range: } (-1, 1) \tag{6.4}$$

Use for: Hidden layers (zero-centered)

6.2.3 ReLU (Rectified Linear Unit) - Most Popular

$$\boxed{f(z) = \max(0, z)}, \quad \text{Output range: } [0, \infty) \tag{6.5}$$

Use for: Hidden layers in deep networks

6.2.4 Leaky ReLU

$$\boxed{f(z) = \max(0.01z, z)}, \quad \text{Output range: } (-\infty, \infty) \tag{6.6}$$

Use for: Preventing dead neurons

6.2.5 Softmax (Multi-class Classification)

$$\boxed{\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}}, \quad \text{Output: Probability distribution (sums to 1)} \tag{6.7}$$

Use for: Multi-class classification output layer

6.3 Forward Propagation

$$\text{Layer 1:} \quad a^{(1)} = f(W^{(1)}x + b^{(1)}) \tag{6.8}$$

$$\text{Layer 2:} \quad a^{(2)} = f(W^{(2)}a^{(1)} + b^{(2)}) \tag{6.9}$$

$$\text{Output Layer:} \quad \hat{y} = f(W^{(L)}a^{(L-1)} + b^{(L)}) \tag{6.10}$$

In matrix form:

$$\boxed{a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})} \tag{6.11}$$

6.4 Backpropagation (Learning from Errors)

Theorem

Using the Chain Rule:

$$\boxed{\frac{\partial J}{\partial w^{(l)}} = \frac{\partial J}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w^{(l)}}} \tag{6.12}$$

Backpropagation Algorithm:

1. Forward pass: Compute predictions $\hat{y}$
2. Compute loss $J(\hat{y}, y)$
3. Compute gradient at output layer
4. Propagate gradients backward through hidden layers
5. Update weights using gradient descent

6.5 Loss Functions for Neural Networks

For Regression:

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 \tag{6.13}$$

For Binary Classification:

$$\text{Binary Cross-Entropy} = -[y\log(\hat{y}) + (1-y)\log(1-\hat{y})] \tag{6.14}$$

For Multi-class Classification:

$$\text{Categorical Cross-Entropy} = -\sum_{j=1}^{K} y_j \log(\hat{y}_j) \tag{6.15}$$

7 Bias-Variance Tradeoff

7.1 Understanding Bias and Variance

Theorem

The Fundamental Tradeoff:

$$\boxed{\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}} \tag{7.1}$$

Definition

- **Bias**: Systematic error from oversimplified model (underfitting)
- **Variance**: Error from sensitivity to small fluctuations in training data (overfitting)
- **Irreducible Error**: Noise inherent in the data (cannot be reduced)

7.2 Underfitting vs Overfitting

Underfitting (High Bias)	**Overfitting (High Variance)**
Model is too simple	Model is too complex
High training error	Very low training error
High test error	High test error
Training and test errors are similar	Large gap between training and test errors
Model fails to capture patterns	Model memorizes noise

Table 7.1: Underfitting vs Overfitting Comparison

7.3 How to Diagnose

Important

Signs of Underfitting:

- Both training and test accuracy are low
- Model performance doesn't improve with more data
- Solution: Increase model complexity, add features, reduce regularization

Signs of Overfitting:

- Training accuracy is very high, test accuracy is low
- Model performance improves with more data
- Solution: Get more data, add regularization, reduce model complexity, early stopping

7.4 Finding the Sweet Spot

```
Error
↑

Total Error
/ \
/ \
/ \
Bias / \ Variance
/ \
/ \
/ \
⟶ ModelComplexity
(Simple) (Optimal) (Complex)
```

Figure 7.1: Bias-Variance Tradeoff Curve

The optimal model balances bias and variance.

8 Unsupervised Learning

8.1 K-Means Clustering

8.1.1 Objective Function

$$\boxed{J = \sum_{i=1}^{m} \sum_{k=1}^{K} r_{ik} \|x^{(i)} - \mu_k\|^2} \tag{8.1}$$

where $r_{ik} = 1$ if point i belongs to cluster k, otherwise 0.

8.1.2 Euclidean Distance

$$\boxed{d(x, \mu) = \sqrt{\sum_{j=1}^{n} (x_j - \mu_j)^2}} \tag{8.2}$$

8.1.3 K-Means Algorithm

1. Choose K (number of clusters)
2. Randomly initialize centroids $\mu_1, \mu_2, \ldots, \mu_K$
3. Repeat until convergence:
   a. Assign each data point to nearest centroid: $c^{(i)} = \arg\min_k \|x^{(i)} - \mu_k\|^2$
   b. Update centroids: $\mu_k = \dfrac{1}{|C_k|} \sum_{x \in C_k} x$

8.1.4 Choosing K - Elbow Method

Compute Within-Cluster Sum of Squares (WCSS):

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2 \tag{8.3}$$

Choose K at the "elbow point" where WCSS stops decreasing rapidly.

8.2 Principal Component Analysis (PCA)

8.2.1 Purpose of PCA

- Reduce number of features (dimensionality reduction)
- Preserve maximum variance (information) in the data
- Visualize high-dimensional data
- Speed up other algorithms

8.2.2 PCA Algorithm

1. Mean-center the data: $X' = X - \mu$
2. Compute covariance matrix: $\Sigma = \frac{1}{m} X'^T X'$
3. Find eigenvalues (λ) and eigenvectors (u): $\Sigma u = \lambda u$
4. Select top k eigenvectors
5. Project data: $X_{\text{reduced}} = X' \times U_k$

8.2.3 Variance Explained

$$\boxed{\text{Variance Explained Ratio} = \frac{\lambda_k}{\sum_{i=1}^{n} \lambda_i}} \tag{8.4}$$

Important

Choosing k:

- Choose smallest k such that cumulative variance $\geq 95\%$
- $\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i} \geq 0.95$

9 Model Evaluation Metrics

9.1 Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

Table 9.1: Confusion Matrix

9.2 Classification Metrics

9.2.1 Accuracy

$$\boxed{\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}} \tag{9.1}$$

Proportion of correct predictions.

9.2.2 Precision

$$\boxed{\text{Precision} = \frac{TP}{TP + FP}} \tag{9.2}$$

Of all positive predictions, how many were correct?

9.2.3 Recall (Sensitivity)

$$\boxed{\text{Recall} = \frac{TP}{TP + FN}} \tag{9.3}$$

Of all actual positives, how many did we catch?

9.2.4 F1-Score (Harmonic Mean)

$$\boxed{F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}} \tag{9.4}$$

Balances precision and recall.

9.2.5 Specificity

$$\boxed{\text{Specificity} = \frac{TN}{TN + FP}} \tag{9.5}$$

Of all actual negatives, how many did we correctly identify?

9.3 Regression Metrics

9.3.1 Mean Squared Error (MSE)

$$\boxed{\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2} \tag{9.6}$$

Penalizes large errors heavily.

9.3.2 Root Mean Squared Error (RMSE)

$$\boxed{\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2}} \tag{9.7}$$

Same units as target variable.

9.3.3 Mean Absolute Error (MAE)

$$\boxed{\text{MAE} = \frac{1}{m}\sum_{i=1}^{m}|\hat{y}_i - y_i|} \tag{9.8}$$

More robust to outliers than MSE.

9.3.4 R-squared (Coefficient of Determination)

$$\boxed{R^2 = 1 - \frac{\sum_{i=1}^{m}(\hat{y}_i - y_i)^2}{\sum_{i=1}^{m}(y_i - \bar{y})^2}} \tag{9.9}$$

Important

Interpreting R^2:

$R^2 = 1.0$	Perfect model (no errors)
$R^2 = 0.85$	Model explains 85% of variance
$R^2 = 0$	Model performs like predicting the mean
$R^2 < 0$	Model performs worse than predicting the mean

9.3.5 Adjusted R-squared (for multiple features)

$$\boxed{R^2_{\text{adj}} = 1 - \frac{(1 - R^2)(m - 1)}{m - n - 1}} \tag{9.10}$$

Penalizes adding unnecessary features.

10 Complete Summary - Quick Reference Card

10.1 Linear Regression

$$\boxed{h_\theta(x) = \theta^T x} \tag{10.1}$$

$$\boxed{J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2} \tag{10.2}$$

10.2 Logistic Regression

$$\boxed{h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}} \tag{10.3}$$

$$\boxed{J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))]} \tag{10.4}$$

10.3 Gradient Descent

$$\boxed{\theta_j := \theta_j - \alpha\frac{\partial}{\partial\theta_j}J(\theta)} \tag{10.5}$$

10.4 Regularization

Ridge (L2)	$J(\theta) = \text{MSE} + \lambda\sum\theta_j^2$
Lasso (L1)	$J(\theta) = \text{MSE} + \lambda\sum\lvert\theta_j\rvert$
Elastic Net	$J(\theta) = \text{MSE} + \lambda_1\sum\lvert\theta_j\rvert + \lambda_2\sum\theta_j^2$

10.5 Neural Networks

$$z = \sum w_i x_i + b \tag{10.6}$$

$$\text{ReLU: } f(z) = \max(0, z) \tag{10.7}$$

$$\text{Sigmoid: } f(z) = \frac{1}{1 + e^{-z}} \tag{10.8}$$

$$\text{Softmax: } f(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \tag{10.9}$$

10.6 Evaluation Metrics - Quick Reference

Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
MSE	$\frac{1}{m} \sum (\hat{y} - y)^2$
RMSE	$\sqrt{\frac{1}{m} \sum (\hat{y} - y)^2}$
MAE	$\frac{1}{m} \sum \lvert \hat{y} - y \rvert$
R^2	$1 - \frac{\sum (\hat{y} - y)^2}{\sum (y - \bar{y})^2}$

10.7 Unsupervised Learning

$$\text{Euclidean Distance: } d(x, \mu) = \sqrt{\sum_{j=1}^{n} (x_j - \mu_j)^2} \tag{10.10}$$

$$\text{K-Means Objective: } J = \sum_{i=1}^{m} \sum_{k=1}^{K} r_{ik} \|x^{(i)} - \mu_k\|^2 \tag{10.11}$$

$$\text{PCA: } \Sigma u = \lambda u \tag{10.12}$$

11 Appendix: Common Machine Learning Algorithms

11.1 Algorithm Cheat Sheet

Algorithm	**Type**	**When to Use**
Linear Regression	Regression	Simple relationships, interpretability needed
Logistic Regression	Classification	Binary classification, probability needed
Decision Trees	Both	Interpretability, non-linear relationships
Random Forest	Both	High accuracy, handles non-linearity well
SVM	Both	Clear margin of separation, high-dimensional data
K-Nearest Neighbors	Both	Small datasets, no training time
Naive Bayes	Classification	Text classification, independence assumptions
K-Means	Clustering	Well-separated spherical clusters
PCA	Dimension Reduction	Visualization, noise reduction
Neural Networks	Both	Complex patterns, large datasets, images, text

Table 11.1: Machine Learning Algorithm Selection Guide

Publication Information

MACHINE LEARNING Complete Course
Logic, Theory, and Equations - From Beginner to Advanced

Author: AUNG KO KO OO
Institute: Burmese NLP Research Institute
Address: Yangon, Myanmar
Email: AungKoKoOo@my.uopeople.edu
www.burmese-nlp-research.org

First Edition: May 7, 2026
ISBN: 978-99971-0-000-0 (Pending)

License: Creative Commons Attribution-NonCommercial 4.0 International

---

This book is dedicated to the advancement of Machine Learning education in Myanmar and the Burmese NLP research community.

Acknowledgments:
Special thanks to all researchers and students who contributed to the development of this educational material.

Disclaimer:
While every effort has been made to ensure accuracy, the author assumes no responsibility for errors or omissions.

References

1. Andrew Ng, "Machine Learning Yearning", deeplearning.ai, 2018.
2. Ian Goodfellow, Yoshua Bengio, Aaron Courville, "Deep Learning", MIT Press, 2016.
3. Christopher M. Bishop, "Pattern Recognition and Machine Learning", Springer, 2006.
4. Trevor Hastie, Robert Tibshirani, Jerome Friedman, "The Elements of Statistical Learning", Springer, 2009.
5. Aurélien Géron, "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow", O'Reilly, 3rd Edition, 2022.
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, "An Introduction to Statistical Learning", Springer, 2nd Edition, 2021.
7. Francois Chollet, "Deep Learning with Python", Manning Publications, 2nd Edition, 2021.
8. Jake VanderPlas, "Python Data Science Handbook", O'Reilly, 2016.